Ranger

Sistema de monitoreo y predicción de fallos

Documentación técnica del backend

Anyi Rafelina Rodríguez Dotel (2024-0022)
Josué Fondeur Román (2024-0193)

Docentes: Willis Ezequiel Polanco · Eduandy Isabel Cruz
Instituto Tecnológico de Las Américas (ITLA)

Octubre 2026

Resumen

Este documento describe el backend de Ranger tal como quedó implementado en la semana 4 del proyecto: la API REST construida con FastAPI, la capa de servicios con la lógica de negocio, el modelo de datos sobre PostgreSQL 16, la seguridad basada en JWT y API key, el worker de análisis que calcula el riesgo y genera alertas inteligentes, y el recolector que alimenta el sistema con métricas y eventos. Para cada componente se explica su responsabilidad, los archivos que lo implementan y las decisiones de diseño más relevantes.

Índice

1. Introducción **3**
1.1. Alcance de esta entrega . . . 3
1.2. Tecnologías . . . 3

2. Arquitectura **3**
2.1. Contenedores . . . 3
2.2. Capas del backend . . . 4

3. Estructura del código **4**

4. Configuración y base de datos **5**
4.1. Configuración (`config.py`) . . . 5
4.2. Conexión (`database.py`) . . . 5
4.3. Migraciones (Alembic) . . . 6
4.4. Seed inicial (`services/seed.py`) . . . 6

5. Modelo de datos **6**

6. Schemas (Pydantic) **7**

7. Seguridad **7**
7.1. Contraseñas y JWT . . . 7

7.2. Dependencias de autorización (`security/deps.py`) . . . 7
7.3. API key de recolectores (`security/api_key.py`) . . . 8

8. Controladores (routers) 8
8.1. Autenticación (`auth.py`) . . . 8
8.2. Usuarios (`users.py`) . . . 8
8.3. Hosts (`hosts.py`) . . . 8
8.4. Ingesta (`ingest.py`) . . . 9
8.5. Métricas (`metrics.py`) . . . 9
8.6. Eventos (`events.py`) . . . 9
8.7. Reglas de alerta (`alert_rules.py`) . . . 9
8.8. Alertas (`alerts.py`) . . . 9
8.9. Incidentes (`incidents.py`) . . . 9
8.10. Predicciones y modelo (`predictions.py`) . . . 10
8.11. Dashboard y reportes (`dashboard.py`, `reports.py`) . . . 10

9. Servicios 10
9.1. Fingerprint de eventos . . . 11

10. Worker de análisis y motor de alertas 11
10.1. Planificación (`worker/scheduler.py`) . . . 11
10.2. Ciclo por host (`worker/jobs.py`) . . . 11
10.3. Pipeline de features (`ml/src/features.py`) . . . 12
10.4. Score de riesgo (`services/risk.py`) . . . 12
10.5. Evaluación de reglas (`services/alert_engine.py`) . . . 13
10.6. Tipos de señal . . . 13
10.7. Situaciones y acciones sugeridas (`services/correlation.py`) . . . 13
10.8. Ciclo de vida de una alerta . . . 14
10.9. Estado del host . . . 14
10.10. Carga del modelo (`worker/model.py`) . . . 14

11. Recolector 14

12. Manejo de errores 15

13. Pruebas 15

14. Ejecución 16

15. Decisiones de diseño relevantes 17
15.1. Próximos pasos . . . 17

1. Introducción

Ranger recolecta métricas, logs y eventos de hosts; los almacena; detecta anomalías con reglas y aprendizaje automático; estima un nivel de riesgo y genera **alertas priorizadas con evidencia y acción sugerida**. El backend es la pieza central del sistema: es el único componente que escribe datos de dominio provenientes del exterior y el que expone toda la información al frontend.

1.1. Alcance de esta entrega

- API REST completa según el contrato `05-API-CONTRATOS.md` (prefijo `/api/v1`).
- Modelo de datos completo de `04-MODELO-DE-DATOS.md`, versionado con Alembic.
- Autenticación JWT, autorización por rol y autenticación de recolectores por API key.
- Ingesta por lotes de métricas y eventos.
- Consultas: métricas crudas y agregadas, eventos con búsqueda, alertas, reglas, incidentes, dashboard, predicciones y reportes con exportación CSV.
- Worker de análisis periódico: features, score de riesgo, motor de alertas con deduplicación, cooldown, escalado y autorresolución.
- Recolector con escenarios de fallo simulados.
- 84 pruebas automatizadas con 96 % de cobertura.

Queda fuera de esta entrega el entrenamiento del modelo de Machine Learning (semanas 6–7). El worker ya está preparado para cargarlo; mientras no exista, opera con reglas y z-score estadístico.

1.2. Tecnologías

Componente	Tecnología
Lenguaje	Python 3.11
Framework web	FastAPI 0.115 + Uvicorn
ORM y migraciones	SQLAlchemy 2.0 (estilo `Mapped`) + Alembic
Base de datos	PostgreSQL 16 con la extensión `pg_trgm`
Validación	Pydantic v2 + pydantic-settings
Seguridad	python-jose (JWT HS256), bcrypt
Procesamiento	pandas, NumPy, scikit-learn, joblib
Planificación	APScheduler (`BlockingScheduler`)
Pruebas y calidad	pytest, httpx (`TestClient`), pytest-cov, ruff
Infraestructura	Docker + Docker Compose, GitHub Actions

2. Arquitectura

2.1. Contenedores

En desarrollo el sistema se levanta con Docker Compose y cuatro servicios del backend:

Servicio	Contenedor	Responsabilidad
`db`	`ranger-db`	PostgreSQL 16, volumen `pgdata`, ejecuta `DB/init.sql`.
`backend`	`ranger-backend`	Aplica las migraciones y sirve la API en el puerto 8000.
`worker`	`ranger-worker`	Misma imagen que el backend; ejecuta el ciclo de análisis cada 60 s.
`collector`	`ranger-collector`	Agente psutil que envía métricas y eventos a la API.

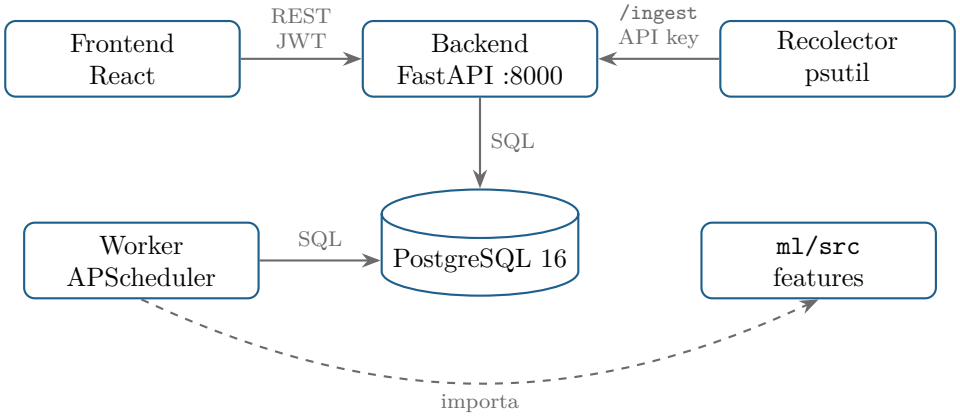


Figura 1: Contenedores y flujos principales.

2.2. Capas del backend

El código sigue una arquitectura por capas. Cada capa solo conoce a la siguiente:

1. **Routers (controladores)** — `BACK/app/routers/`. Declaran los endpoints, los parámetros, el schema de respuesta y la dependencia de rol. No contienen lógica de negocio ni consultas SQL: delegan en un servicio.
2. **Schemas** — `BACK/app/schemas/`. Modelos Pydantic de entrada y salida (`XxxCreate`, `XxxUpdate`, `XxxRead`). Validan tipos, rangos y tamaños de lote.
3. **Servicios** — `BACK/app/services/`. Lógica de negocio y consultas con el ORM. Lanzan `AppError` con un código de error de dominio.
4. **Modelos** — `BACK/app/models/`. Entidades SQLAlchemy que reflejan exactamente el DDL de `04`.
5. **Seguridad** — `BACK/app/security/`. Hashing, JWT, dependencias de rol y API key; se inyectan con `Depends`.

Flujo de una petición típica:

```
Cliente -> Router (valida schema + rol) -> Servicio (lógica + ORM)
        -> PostgreSQL -> Servicio -> Router (response_model) -> JSON
```

3. Estructura del código

```
BACK/
|-- Dockerfile              # imagen python:3.11-slim; migra y arranca uvicorn
|-- requirements.txt
|-- alembic.ini
|-- alembic/
|   |-- env.py              # toma la URL de app.config
|   `-- versions/           # migración inicial (esquema completo de 04)
`-- app/
    |-- main.py             # FastAPI, CORS, /health, lifespan (seed)
    |-- config.py           # Settings (pydantic-settings)
    |-- database.py         # engine, SessionLocal, Base, get_db
    |-- errors.py           # AppError y manejadores {detail, code}
    |-- models/             # 9 entidades + enums
    |-- schemas/            # common, user, host, metric, ingest, query, prediction
    |-- routers/            # 12 routers
    |-- services/           # 17 módulos de lógica de negocio
    |-- security/           # hashing, jwt, deps, api_key
    `-- worker/             # scheduler, jobs, model
ml/src/features.py          # pipeline de features compartido
COLLECTOR/                  # collector.py, scenarios.py
TEST/                       # conftest, factories, api/, back/, ml/, integration/
```

4. Configuración y base de datos

4.1. Configuración (`config.py`)

La clase `Settings` lee las variables de entorno (archivo `.env`). Los secretos **no tienen valor por defecto**: si faltan, la aplicación no arranca. Se accede siempre mediante `get_settings()`, que se cachea con `lru_cache`.

Variable	Defecto	Uso
`DATABASE_URL`	—	Conexión SQLAlchemy (`postgresql+psycopg://...`).
`TEST_DATABASE_URL`	derivada	BD de pruebas; por defecto `<db>_test`.
`APP_ENV`	`development`	Nivel de log (DEBUG en desarrollo).
`CORS_ORIGINS`	`localhost:5173`	Orígenes permitidos, separados por coma.
`JWT_SECRET`	—	Clave de firma de los tokens.
`JWT_EXPIRE_MINUTES`	480	Vigencia del token.
`COLLECTOR_API_KEY`	—	Clave que deben enviar los recolectores.
`ADMIN_EMAIL`, `ADMIN_PASSWORD`	—	Usuario administrador inicial.
`ANALYSIS_INTERVAL_SECONDS`	60	Periodo del ciclo del worker.
`ALERT_COOLDOWN_MINUTES`	15	Cooldown tras resolver una alerta.
`COLLECT_INTERVAL`	10	Intervalo del recolector; define cuándo un host está caído.
`MODEL_DIR`	`/code/ml/models`	Carpeta de artefactos `.joblib`.

4.2. Conexión (`database.py`)

Define el `engine` (con `pool_pre_ping`), la fábrica `SessionLocal`, la clase `Base` declarativa (que mapea `datetime` a `timestamptz`) y la dependencia `get_db()`, que abre una sesión por petición y la cierra al terminar.

4.3. Migraciones (Alembic)

El esquema lo crea Alembic (ADR-07 aceptada). La migración inicial se generó con `-autogenerate` a partir de los modelos y se completó a mano con la creación de la extensión `pg_trgm` y el borrado de los tipos `ENUM` en el `downgrade`. El contenedor `backend` ejecuta `alembic upgrade head` antes de iniciar Uvicorn. `DB/init.sql` solo crea la extensión.

Una prueba (`test_migration_matches_models`) compara la base migrada con los modelos mediante `compare_metadata`: si alguien cambia un modelo sin crear la migración, la prueba falla.

4.4. Seed inicial (`services/seed.py`)

Al arrancar la API (`lifespan` de `main.py`) se ejecuta `seed_defaults`, que es idempotente:

- crea el usuario `admin` con `ADMIN_EMAIL`/`ADMIN_PASSWORD` si no existe;
- crea las cinco reglas por defecto si la tabla `alert_rules` está vacía.

Regla	Métrica	Op.	Umbral	Dur.	Sev.
CPU alta sostenida	`cpu_percent`	$>$	90	5	high
RAM alta	`mem_percent`	$>$	85	0	medium
Disco libre bajo	`disk_free_percent`	$<$	10	0	high
Riesgo del modelo alto	`risk_score`	$\geq$	0.6	0	high
Ráfaga de errores	`events_error_count`	$\geq$	20	5	medium

5. Modelo de datos

Los modelos (`BACK/app/models/`) usan SQLAlchemy 2.0 con `Mapped[...]` y `mapped_column`, y reflejan exactamente el DDL de `04`: nombres, tipos (`INET`, `REAL[]`, `JSONB`, `DOUBLE PRECISION`), valores por defecto e índices, incluidos el índice trigram de mensajes y el índice único parcial de deduplicación.

Clase	Tabla	Descripción
`User`	`users`	Usuarios con rol, estado y último acceso.
`Host`	`hosts`	Servidores, aplicaciones, servicios o bases de datos monitoreados.
`Metric`	`metrics`	Una fila por lectura del recolector (CPU, RAM, disco, red, latencia, errores, disponibilidad).
`Event`	`events`	Logs y eventos con nivel, origen, mensaje y `fingerprint`.
`AlertRule`	`alert_rules`	Reglas de umbral configurables, globales o por host.
`ModelRegistry`	`model_registry`	Versiones del modelo ML con sus métricas de evaluación.
`Prediction`	`predictions`	Resultado de cada ciclo del worker por host.
`Alert`	`alerts`	Alertas con evidencia, tendencia, riesgo, acción sugerida y `dedup_key`.
`Incident`	`incidents`	Incidentes con causa raíz y resolución.
—	`incident_alerts`	Tabla puente N:M entre incidentes y alertas.

Enums. `enums.py` define `UserRole`, `HostKind`, `HostStatus`, `EventLevel`, `SeverityLevel`, `AlertStatus`, `AlertOrigin`, `RuleOperator` e `IncidentStatus` como `str, Enum`, mapeados a los tipos `ENUM` de PostgreSQL. El orden de `severity_level` (low < medium < high < critical) permite ordenar por severidad directamente en SQL. `SEVERITY_ORDER` da ese mismo orden en Python para el escalado.

Propiedades de conveniencia. `Alert`, `Event`, `Prediction` e `Incident` exponen una propiedad `hostname` a partir de su relación con `Host`, para que los schemas de respuesta la incluyan sin consultas manuales.

6. Schemas (Pydantic)

Módulo	Contenido
`common.py`	`Page[T]` (`items, total, page, size`) y el tipo `Email` (normalizado a minúsculas).
`user.py`	`UserCreate`, `UserUpdate`, `UserRead`, `Token`. La contraseña nunca se devuelve.
`host.py`	`HostCreate`, `HostUpdate`, `HostRead`, `HostDetail` (con última métrica y último riesgo). `ip_address` se valida como IP y se serializa como texto.
`metric.py`	`MetricReading`, `MetricsBatch` (1 a 500 lecturas), `MetricRead`. Porcentajes en 0–100, valores no negativos; fechas sin zona horaria se interpretan como UTC.
`ingest.py`	`IngestRegister`, `EventIn`, `EventsBatch` (1 a 500 eventos) y las respuestas `{host_id}` y `{accepted}`.
`query.py`	Lectura: `MetricAggregate`, `EventRead`, `AlertRead`, reglas, incidentes, `DashboardOverview` y `AvailabilityReport`.
`prediction.py`	`PredictionLatest`, `PredictionPoint`, `PredictionSummary`, `ModelRead`.

Los schemas de creación incluyen un ejemplo (`json_schema_extra`) que aparece en Swagger (`/docs`).

7. Seguridad

7.1. Contraseñas y JWT

- `security/hashing.py`: `hash_password` y `verify_password` con **bcrypt** (se usa `bcrypt` directamente porque `passlib` está sin mantenimiento y falla con bcrypt $\geq$ 4.1).
- `security/jwt.py`: `create_access_token` firma un token HS256 con `sub` (id del usuario), `role` y `exp`. `decode_access_token` devuelve `None` si el token es inválido o expiró.

7.2. Dependencias de autorización (`security/deps.py`)

`get_current_user` extrae el token `Bearer`, lo valida y carga el usuario: si falta, es inválido o el usuario está inactivo, responde `401`. `require_roles(*roles)` devuelve una dependencia que responde `403` si el rol del usuario no está permitido. Los grupos de roles implementan la matriz de permisos de `05`:

Constante	Roles	Uso típico
`ADMIN`	admin	Usuarios, activar modelo, borrar hosts
`ADMIN_INFRA`	admin, infra	Escritura de hosts, reglas, alertas e incidentes
`NOT_VIEWER`	admin, infra, developer, analyst	Métricas, eventos, reglas, predicciones
`EXPORTERS`	admin, infra, analyst	Exportaciones CSV
`REPORT_READERS`	admin, infra, analyst, viewer	Reporte de disponibilidad

7.3. API key de recolectores (`security/api_key.py`)

Todos los endpoints `/ingest/*` exigen la cabecera `X-Collector-Key`. La comparación usa `secrets.compare_digest` para evitar ataques de tiempo. Si la clave falta o no coincide, responde `401 INVALID_API_KEY`.

8. Controladores (routers)

Todos los routers se registran en `routers/__init__.py` bajo el prefijo `/api/v1`. `GET /health` queda fuera del prefijo. Las listas paginadas aceptan `page` y `size` y devuelven `{items, total, page, size}`. Los parámetros `from`/`to` aceptan ISO-8601 o expresiones relativas (`now`, `now-15m`, `now-6h`, `now-7d`).

8.1. Autenticación (`auth.py`)

POST `/auth/login` público Formulario `username` (email) + `password` → `{access_token, token_type, user}`; actualiza `last_login_at`.
GET `/auth/me` autenticado Usuario actual.

8.2. Usuarios (`users.py`)

GET `/users` admin Listado paginado.
POST `/users` admin Crea usuario; `409 EMAIL_TAKEN` si el email existe.
PATCH `/users/{user_id}` admin Cambia nombre, rol o `is_active`.
DELETE `/users/{user_id}` admin Desactivación lógica (soft delete).
Un administrador no puede desactivarse ni cambiar su propio rol (`400 SELF_MODIFICATION`).

8.3. Hosts (`hosts.py`)

GET `/hosts` autenticado Paginado; filtros `environment`, `area`, `status`, `kind`, `is_active`.
GET `/hosts/{host_id}` autenticado Detalle + última métrica + última predicción.
POST `/hosts` admin, infra Registro manual; `409 HOSTNAME_TAKEN`.
PATCH `/hosts/{host_id}` admin, infra Edita metadatos.
DELETE `/hosts/{host_id}` admin Desactiva el host.

8.4. Ingesta (`ingest.py`)

POST `/ingest/register` API key Upsert del host por `hostname` → `{host_id}`.
POST `/ingest/metrics` API key Lote de lecturas (máx. 500) → `202 {accepted}`.
POST `/ingest/events` API key Lote de eventos (máx. 500) → `202 {accepted}`.

8.5. Métricas (`metrics.py`)

GET `/metrics` no viewer Lecturas crudas en orden cronológico; `host_id`, `from`, `to`, `limit` $\leq$ 5000.
GET `/metrics/latest` no viewer Última lectura por host activo.
GET `/metrics/aggregate` no viewer `metric`, `bucket` (minute, hour, day) → `[{period, avg, min, max, samples, alerts}]`.
GET `/metrics/export.csv` exporters Mismo agregado en CSV descargable.

8.6. Eventos (`events.py`)

GET `/events` no viewer Búsqueda `q` (ILIKE con índice trigram), `host_id`, `level` (repetible), `source`, `from`, `to`; paginado.
GET `/events/summary` no viewer `{info, warning, error, critical}` con los mismos filtros salvo `level`.

8.7. Reglas de alerta (`alert_rules.py`)

GET `/alert-rules` no viewer Lista de reglas.
POST `/alert-rules` admin, infra Nueva regla; valida la métrica y que el host exista.
PATCH `/alert-rules/{rule_id}` admin, infra Edita una regla.
DELETE `/alert-rules/{rule_id}` admin, infra Elimina una regla (`204`).

8.8. Alertas (`alerts.py`)

GET `/alerts` autenticado Filtros `status`, `severity`, `origin`, `host_id`, rango; orden por severidad, riesgo y fecha.
GET `/alerts/summary` autenticado Conteo de alertas activas por severidad.
GET `/alerts/{alert_id}` autenticado Detalle con `evidence`, `trend`, `situation`, `suggested_action`.
POST `/alerts/{alert_id}/acknowledge` admin, infra Marca `acknowledged`; `400` si ya está resuelta.
POST `/alerts/{alert_id}/resolve` admin, infra Marca `resolved`.

8.9. Incidentes (`incidents.py`)

GET `/incidents` autenticado Filtros `status`, `severity`, `host_id`; paginado.
POST `/incidents` admin, infra Crea un incidente; `alert_ids` opcional para vincular alertas.
GET `/incidents/{incident_id}` autenticado Detalle con alertas vinculadas.
PATCH `/incidents/{incident_id}` admin, infra Estado, causa raíz, resolución, asignado,

alertas.

8.10. Predicciones y modelo (`predictions.py`)

GET `/predictions/latest` no viewer Última predicción por host.
GET `/predictions` no viewer Serie de riesgo en el tiempo (24 h por defecto).
GET `/predictions/summary` autenticado `{current_risk_level, risk_score, failure_probability, anomalies_today}`.
GET `/models/active` autenticado Modelo activo o `null`.
POST `/models/{model_id}/activate` admin Activa una versión y desactiva las demás.

8.11. Dashboard y reportes (`dashboard.py`, `reports.py`)

GET `/dashboard/overview` autenticado `host_id`, `range` (1h, 6h, 24h, 7d) → KPIs con tendencia, serie, riesgo global, 5 alertas recientes y hosts.
GET `/reports/availability` report readers Disponibilidad % por host y MTTR de incidentes (7 días por defecto).
GET `/reports/availability.csv` exporters Exportación CSV.

9. Servicios

Los servicios concentran la lógica de negocio. Reciben la sesión de base de datos y datos ya validados, y devuelven entidades ORM o diccionarios que el router serializa.

Servicio	Responsabilidad
`users.py`	Autenticación (email normalizado, usuario activo, verificación bcrypt), CRUD y reglas contra la auto-modificación.
`hosts.py`	CRUD de hosts, búsqueda por hostname y detalle con la última métrica y la última predicción.
`ingest.py`	Upsert del host al registrarse; inserción por lotes de métricas y eventos; actualiza `last_seen_at` (hora del servidor) y marca `up`/`down` según `is_available`. Calcula el `fingerprint` de cada evento.
`metrics.py`	Lecturas crudas, última lectura por host (`DISTINCT ON`), agregación con `date_trunc` y conteo de alertas por periodo, generación de CSV.
`events.py`	`EventFilters` reutilizable: búsqueda ILIKE con escape de comodines, filtro multinivel, paginación y conteo por nivel.
`alert_rules.py`	CRUD de reglas con validación del host.
`alerts.py`	Listado priorizado, resumen por severidad, reconocer y resolver.
`incidents.py`	Alta y edición de incidentes, vínculo con alertas, deducción del host y gestión de `closed_at`.
`dashboard.py`	KPIs promedio, serie con `date_bin` (30 puntos), riesgo global (peor predicción), alertas recientes y estado de hosts.
`reports.py`	Disponibilidad (`AVG(is_available)`), MTTR (`closed_at - opened_at`) y CSV.

Servicio	Responsabilidad
`predictions.py`	Serie de predicciones, resumen, modelo activo y activación de versión.
`timeutil.py`	Parseo de fechas absolutas y relativas, dependencia `TimeRange` y rangos del dashboard.
`pagination.py`	`paginate()`: conteo total con subconsulta, `LIMIT/OFFSET`.
`seed.py`	Admin y reglas por defecto.
`risk.py`	Fórmula del score de riesgo, niveles, escalado y ajuste por contexto.
`correlation.py`	Evidencia legible, tendencia, situaciones de cuello de botella y acciones sugeridas.
`alert_engine.py`	Evaluación de reglas, señales y ciclo de vida de las alertas.

9.1. Fingerprint de eventos

Para agrupar mensajes repetidos, `normalize_message` pasa el texto a minúsculas y reemplaza UUIDs, direcciones IP, valores hexadecimales y números por marcadores (`<uuid>`, `<ip>`, `<hex>`, `<n>`). El fingerprint es el SHA-256 de `source + "|" + mensaje normalizado`. Así, *"HTTP 503 GET /orders/12"* y *"HTTP 503 GET /orders/99"* quedan en el mismo grupo.

10. Worker de análisis y motor de alertas

10.1. Planificación (`worker/scheduler.py`)

El worker corre como un proceso independiente (`python -m app.worker.scheduler`) con un `BlockingScheduler` de APScheduler y dos jobs:

- `analyze_all_hosts`: cada `ANALYSIS_INTERVAL_SECONDS`, sin ejecuciones superpuestas (`max_instances=1`, `coalesce=True`).
- `cleanup`: diario a las 03:00 UTC; borra predicciones con más de 30 días.

10.2. Ciclo por host (`worker/jobs.py`)

La clase `Analyzer` recorre los hosts activos. Cada host se procesa en un `try/except` propio: si uno falla, se registra el error, se hace `rollback` y el ciclo continúa con los demás.

1. **Carga**: métricas de las últimas 24 h agregadas por minuto en SQL y eventos de los últimos 60 min.
2. **Datos frescos**: si no hay lecturas en los últimos 5 min, no se predice; solo se actualiza el estado y se autorresuelven alertas.
3. **Features**: `build_features` (ver §10.3).
4. **Reglas de métricas**: `evaluate_metric_rules`.
5. **Scoring**: modelo activo o, si no existe, z-score estadístico.
6. **Riesgo**: `combine` + `risk_level` + ajuste por contexto.
7. **Predicción**: `INSERT` en `predictions` con el vector de features.
8. **Reglas de riesgo**: `evaluate_risk_rules` con el historial reciente.
9. **Señales**: regla, modelo y correlación.

10. **Alertas**: `apply_signals` y `auto_resolve`.
11. **Estado del host**: `up`, `degraded`, `down` o `unknown`.

Al final del ciclo, `correlate_areas` busca áreas con señales en dos o más hosts.

10.3. Pipeline de features (`ml/src/features.py`)

La misma función se usa en el worker y se usará en el entrenamiento, para evitar diferencias entre ambos (*training/serving skew*).

1. **Limpieza**: porcentajes recortados a 100 y valores negativos descartados.
2. **Remuestreo** a 1 minuto con *forward-fill* de hasta 2 min; los huecos mayores se marcan como `is_available = 0`.
3. **Cálculo de features** por minuto:

Grupo	Features
Nivel actual	`cpu_percent`, `mem_percent`, `disk_percent`, `net_recv_bps`, `net_sent_bps`, `latency_ms`, `error_rate`
Rolling	`mean`, `std`, `max` a 5, 15 y 60 min de CPU, RAM y latencia
Tendencia	`slope_15m`, `slope_60m` de RAM, disco y latencia (regresión lineal vectorizada)
Desviación	`zscore_60m`: valor actual frente a la media y desviación de 24 h (mín. 60 muestras)
Eventos	`err_count_5m`, `crit_count_5m`, `warn_count_15m`, `distinct_fingerprints_15m`
Disponibilidad	`downtime_minutes_60m`
Contexto	`hour_sin`, `hour_cos`

La pendiente se calcula sin bucles con medias móviles:

$$\text{slope} = \frac{\overline{xy} - \bar{x}\,\bar{y}}{\overline{x^2} - \bar{x}^2},$$

donde x es la posición del minuto, enmascarada donde no hay dato.

10.4. Score de riesgo (`services/risk.py`)

$$\text{risk} = 0.45 \cdot P(\text{fallo}) + 0.35 \cdot \text{anomalía} + 0.20 \cdot \text{reglas}$$

- **reglas**: máxima severidad de las reglas disparadas (low 0.25, medium 0.5, high 0.75, critical 1).
- **Sin clasificador**: $P(\text{fallo})$ no existe y los pesos se renormalizan: $\text{risk} = (0.35 \cdot \text{anomalía} + 0.20 \cdot \text{reglas})/0.55$.
- **Anomalía sin modelo**: $\text{mín}(1, \text{máx}\,|z|/4)$; se considera anomalía si $|z| \geq 3$.

risk_score	Nivel
< 0.30	low (Bajo)
0.30 – 0.59	medium (Medio)
0.60 – 0.79	high (Alto)
≥ 0.80	critical (Crítico)

Ajuste por contexto: en hosts con `environment = prod`, si el riesgo se mantiene en medio o superior durante tres ciclos seguidos, se sube un nivel.

10.5. Evaluación de reglas (`services/alert_engine.py`)

- **Métricas** (`cpu_percent`, `mem_percent`, `latency_ms`, `error_rate`, `disk_free_percent` = 100− `disk_percent`): la condición debe cumplirse en *todos* los minutos de los últimos `duration_minutes` (mínimo 1) y sin huecos.
- `events_error_count`: eventos `error` o `critical` en los últimos máx(duración, 5) minutos.
- `risk_score`: el riesgo actual y las predicciones de la ventana deben cumplir la condición.
- Las reglas con `host_id` solo se aplican a ese host; las demás, a todos.

Cada regla disparada genera una `Signal` con título, descripción, evidencia legible (p. ej. *"95 % durante 8 min"*, *"de 180 ms a 620 ms"*), tendencia, situación y acción sugerida.

10.6. Tipos de señal

Origen	Cuándo se genera
`rule`	Una regla se cumple.
`model`	El nivel de riesgo es alto o crítico y no se disparó ya una regla de `risk_score`.
`correlation` (host)	Hay reglas disparadas de dos o más categorías (CPU, RAM, disco, latencia, errores). Con tres o más, la severidad sube un nivel. Incluye los tres errores más frecuentes agrupados por fingerprint.
`correlation` (área)	Dos o más hosts del mismo `area` tienen señales en el mismo ciclo.

10.7. Situaciones y acciones sugeridas (`services/correlation.py`)

Situación	Heurística	Acción sugerida
Saturación de CPU	CPU alta y latencia en aumento	Revisar procesos con mayor consumo y picos de tráfico; evaluar escalar recursos
Posible fuga de memoria	Pendiente de RAM positiva y RAM alta o errores OOM	Revisar consumo por proceso, reinicio controlado, analizar heap
Saturación de almacenamiento	Regla de disco libre; ETA = (100 − disco)/pendiente	Liberar espacio, revisar rotación de logs, ampliar volumen
Dependencia externa	Latencia y errores en aumento con CPU, RAM y disco normales	Revisar dependencias, pool de conexiones y timeouts
Ráfaga de errores	Reglas de errores	Revisar logs y despliegues recientes
Degradación progresiva	Tres o más categorías, o RAM + latencia + errores	Revisar recursos, errores y conexiones del servicio

10.8. Ciclo de vida de una alerta

Deduplicación Clave `dedup_key = host_id:origin:rule_id|mÃ¯trica`. Si ya existe una alerta no resuelta con esa clave, se incrementa `occurrences` y se actualizan `last_seen_at`, la evidencia y el riesgo. Un índice único parcial en PostgreSQL garantiza que no haya dos alertas abiertas con la misma clave.

Escalado Si la nueva severidad es mayor, la alerta sube de severidad. Nunca baja.

Cooldown Si una alerta con la misma clave se resolvió hace menos de `ALERT_COOLDOWN_MINUTES`, no se crea una nueva.

Autorresolución Una alerta cuya señal no aparece desde 2.5× el intervalo del worker (es decir, en el tercer ciclo limpio) pasa a `resolved`.

Manual Los usuarios infra y admin pueden reconocer (`acknowledged`) o resolver alertas desde la API.

10.9. Estado del host

Estado	Condición
`unknown`	El host nunca envió datos.
`down`	Sin contacto durante más de 3× `COLLECT_INTERVAL`, o última lectura no disponible.
`degraded`	Riesgo alto o crítico.
`up`	En cualquier otro caso.

10.10. Carga del modelo (`worker/model.py`)

`ModelManager` consulta en cada ciclo el modelo activo de `model_registry` y solo lo recarga si cambió. El artefacto `.joblib` es un diccionario con:

- `features`: lista ordenada de columnas esperadas;
- `classifier`: estimador con `predict_proba` $\rightarrow P(\text{fallo})$;
- `anomaly_model` (opcional): estimador con `score_samples`, p. ej. IsolationForest, más `anomaly_bounds` para normalizar a 0–1;
- `feature_means` y `feature_stds` (opcionales) para calcular el factor principal (importancia $\times|z|$).

Si el artefacto no existe o falla al cargar o al puntuar, se registra el error y se usa el modo de respaldo (z-score), sin detener el worker.

11. Recolector

`COLLECTOR/collector.py` es un agente independiente que:

1. se registra con `POST /ingest/register` (hostname, sistema operativo, núcleos, memoria, IP);
2. cada `COLLECT_INTERVAL` segundos lee con psutil la CPU (total y por núcleo), la memoria, las particiones de disco, la red (bytes por segundo entre lecturas) y la latencia hacia `/health`;
3. aplica el escenario configurado y envía métricas y eventos en lotes de hasta 500;

4. si la API no responde, guarda los datos en un buffer en memoria (máx. 10 000) y los reenvía en el siguiente ciclo; si recibe `404`, se vuelve a registrar.

`COLLECTOR/scenarios.py` degrada la lectura real de forma progresiva durante `SCENARIO_RAMP_MINUTES` (30 por defecto). Al llegar al 100 % emite un evento `critical` con `source = "scenario"` y `attributes.failure = true` (la etiqueta para el modelo), simula 3 minutos de caída y reinicia.

Escenario	Efecto
`normal`	Lectura real y eventos informativos ocasionales.
`cpu_spike`	CPU hasta 97 % y aumento de latencia.
`memory_leak`	RAM hasta 98 % y errores *OutOfMemoryError*.
`disk_fill`	Disco hasta 99.5 % y errores *No space left on device*.
`error_burst`	`error_rate` hasta 25 % y ráfagas de HTTP 503.
`degradation`	CPU, RAM, latencia (180 → 680 ms) y errores combinados.

12. Manejo de errores

`errors.py` define `AppError` (una `HTTPException` con `code`) y registra manejadores para que *todas* las respuestas de error tengan la forma:

```
{ "detail": "Mensaje legible", "code": "HOST_NOT_FOUND" }
```

HTTP	Códigos	Situación
400	`SELF_MODIFICATION`, `INVALID_HOST`, `INVALID_USER`, `INVALID_ALERT_IDS`, `INVALID_RANGE`, `ALERT_ALREADY_RESOLVED`	Validación de negocio.
401	`UNAUTHORIZED`, `INVALID_CREDENTIALS`, `INVALID_API_KEY`	Sin token, token inválido, credenciales o API key incorrectas.
403	`FORBIDDEN`	Rol sin permiso.
404	`USER_NOT_FOUND`, `HOST_NOT_FOUND`, `ALERT_NOT_FOUND`, `RULE_NOT_FOUND`, `INCIDENT_NOT_FOUND`, `MODEL_NOT_FOUND`	Recurso inexistente.
409	`EMAIL_TAKEN`, `HOSTNAME_TAKEN`	Duplicados.
422	`VALIDATION_ERROR`, `INVALID_DATE`	Error de schema (`detail` conserva la lista de Pydantic) o fecha inválida.

13. Pruebas

Las pruebas usan una base de datos propia (`<POSTGRES_DB>_test`). Se recrea y se migra con Alembic al inicio de cada sesión de pytest, y las tablas se vacían después de cada prueba. `TEST/conftest.py` provee los fixtures `db`, `client`, `make_user`, `auth_headers(role)` y `collector_headers`. `TEST/factories.py` crea hosts, métricas, eventos, alertas y predicciones de prueba.

Archivo	Qué verifica
`test_health.py`	`/health`.
`back/test_schema.py`	Tablas creadas, migración igual a los modelos, índices especiales, seed idempotente.
`back/test_security.py`	bcrypt y JWT (validez, expiración, alteración).
`back/test_fingerprint.py`	Normalización y agrupación de mensajes.
`back/test_scenarios.py`	Rampa, fallo, caída y recuperación de los escenarios.
`back/test_alert_engine.py`	Dedup, escalado, cooldown, autorresolución, duración de reglas, reglas de disco, eventos y riesgo.
`api/test_auth_users.py`	Login, 401, 403, 409, 422 y CRUD de usuarios.
`api/test_hosts_ingest.py`	API key, upsert, lotes, límites, fingerprint, CRUD de hosts.
`api/test_metrics_events.py`	Rangos relativos, agregados, CSV, búsqueda y resumen de eventos.
`api/test_alerts_incidents.py`	Reglas, priorización de alertas, ack/resolve, ciclo de vida de incidentes.
`api/test_dashboard_reports.py`	Dashboard vacío y con datos, disponibilidad y MTTR.
`api/test_predictions_api.py`	Predicciones, resumen, modelo activo y activación.
`ml/test_features.py`	Pendientes, remuestreo, huecos, rolling, z-score y features de eventos.
`ml/test_risk.py`	Umbrales de nivel, fórmula, renormalización, ajuste por contexto.
`ml/test_model_contract.py`	El artefacto carga y produce probabilidades en $[0, 1]$; respaldo si falta.
`integration/test_worker_cycle.py`	Ciclo completo: predicción, alertas correlacionadas, deduplicación, autorresolución, `down`, correlación por área y tolerancia a fallos.

Resultado: 84 pruebas aprobadas. La cobertura total es del 96 %, y la de `app/services` y `ml/src` supera el 90 % (el objetivo de `09` es 70 %). `ruff format` y `ruff check` pasan sin errores. El flujo de GitHub Actions (`.github/workflows/ci.yml`) ejecuta ruff, las migraciones y pytest con un servicio PostgreSQL 16 en cada pull request hacia `develop` o `main`.

14. Ejecución

```
cp .env.example .env              # solo la primera vez
docker compose up -d --build      # db, backend, worker y collector

# Verificación
curl http://localhost:8000/health # {"status":"ok"}
# Swagger: http://localhost:8000/docs   (admin@ranger.local / admin123)

# Pruebas
docker compose exec backend pytest /code/TEST -q

# Logs y parada
docker compose logs -f backend worker collector
docker compose down               # down -v borra también la base de datos
```

Para simular un fallo, se cambia `COLLECTOR_SCENARIO` en `.env` (por ejemplo, `degradation`) y se ejecuta `docker compose up -d collector`.

15. Decisiones de diseño relevantes

El registro completo está en `DOCS/DECISIONES.md` (27 entradas). Las más relevantes:

- **Fechas relativas**: `from`/`to` aceptan `now-15m` además de ISO-8601, como pide la pantalla de Métricas.
- **Dashboard**: agrega el campo `series` para la gráfica de uso de recursos y devuelve `null` en el riesgo mientras no haya predicciones.
- **Permisos**: donde `05` se contradice, se aplicó su matriz de permisos (el viewer no ve reglas ni la serie de predicciones).
- **Riesgo sin modelo**: los pesos se renormalizan para cubrir 0–1.
- **Historial para anomalías**: el z-score necesita al menos 60 minutos de datos; un host nuevo tiene anomalía 0 durante su primera hora (importante al preparar la demo).
- **`last_seen_at`**: es la hora de recepción en el servidor, no la del recolector, para que relojes desfasados o lotes atrasados no afecten el estado.
- **Producción**: el backend importa `ml/` desde un volumen; para la Fase 9 hay que incluirlo en la imagen.

15.1. Próximos pasos

1. Conectar el frontend a la API (hito del 25 de octubre).
2. Generar el dataset histórico con los escenarios y entrenar el modelo v1 (IsolationForest + RandomForest) siguiendo el contrato de artefacto descrito.
3. Registrar el modelo en `model_registry` y validar la demo con el escenario `degradation`.
4. Preparar `docker-compose.prod.yml` con Nginx y secretos de producción.